

CSV Files and Tabular Records

Day 5 • Block 35 • 30 minutes teaching + 30 minutes exercises

From spreadsheet-shaped text to Python records



Key shift

Data still needs loops, functions, validation, and output. The source changed; your Python thinking still applies.

What Students Should Be Able to Do

The goal is not memorizing csv syntax; it is turning rows into records.

1

Explain rows, columns, headers, and delimiters

4

Convert numeric CSV fields from text into numbers

2

Use the csv module instead of splitting manually

5

Write output using csv.writer and csv.DictWriter

3

Read rows with csv.reader and csv.DictReader

6

Validate required columns before processing

CSV Is Table-Shaped Text

A comma-separated file can represent rows and columns.

```
record_id,status,error_count  
EQ-1001,active,0  
EQ-1002,active,3  
EQ-1003,inactive,0
```

same
info

record_id	status	error_count
EQ-1001	active	0
EQ-1002	active	3
EQ-1003	inactive	0

Mental model

CSV is text on disk, but we usually think about it as rows and columns.

Headers Name the Columns

The first row often tells Python what each value means.

header row	record_id,status,error_count
data row	EQ-1001,active,0
data row	EQ-1002,active,3

record_id



status



error_count



Why it matters

A header lets DictReader create dictionaries with meaningful keys instead of position-only lists.

CSV Is More Than split(",")

Quoted text can legally contain commas.

```
line = 'EMP-1001,"Smith, Jordan",active'  
values = line.split(",")  
print(values)
```

Wrong pieces

```
[  
    "EMP-1001",  
    "Smith",  
    " Jordan",  
    "active"  
]
```

Rule

Use a CSV parser for CSV data. The parser understands commas inside quoted values.

Use Python's csv Module

The standard library already knows how to parse CSV correctly.

```
import csv

with open(
    "data/records.csv",
    encoding="utf-8"
) as file:

    reader = csv.reader(file)

    for row in reader:
        print(row)
```

What csv.reader gives you

Each row becomes a list of strings.

```
["EQ-1001", "active", "0"]
```

Remember

The parser solves CSV formatting. It does not know your business rules.

csv.reader Produces Lists

Lists are useful, but column positions matter.

```
for row in reader:  
    record_id = row[0]  
    status = row[1]  
    error_count = row[2]  
  
    print(record_id, status, error_count)
```

Position-based access

row[0] means first column
row[1] means second column
row[2] means third column

Tradeoff

csv.reader is simple, but your code must remember which column number means what.

DictReader Produces Records

Because students already know dictionaries, this is the usual teaching path.

```
import csv

with open(
    "data/records.csv",
    encoding="utf-8"
) as file:

    reader = csv.DictReader(file)

    for record in reader:
        print(record)
```

One row becomes one dictionary

```
{
    "record_id": "EQ-1001",
    "status": "active",
    "error_count": "0",
}
```

Connection

A CSV row can now flow into the Day 3 and Day 4 record-processing functions.

CSV Row → Dictionary → Function

This is where earlier course pieces start to connect.

Why functions matter

The input source changed from a hard-coded list to a file. The processing logic can stay reusable.



```
for record in reader:  
    processed = process_record(record)  
    processed_records.append(processed)
```

CSV Values Arrive as Text

A numeric-looking CSV value is still read as a string.

From the CSV file

```
record_id,status,error_count  
EQ-1001,active,0
```

Inside the dictionary

```
record["error_count"]  
  
# value is "0"  
# not 0
```

Practical rule

Convert before numeric comparisons, calculations, or classifications.

Convert Numeric Fields Explicitly

Python cannot compare strings and numbers the way you intend.

```
record = {  
    "record_id": "EQ-1001",  
    "error_count": "3",  
    "completion": "95",  
}  
  
error_count = int(record["error_count"])  
completion = int(record["completion"])
```

After conversion

```
error_count    # 3  
completion     # 95
```

Day 2 connection

Files bring back type conversion. Input arrives as text until your code converts it.

Normalize Before You Classify

Whitespace and capitalization are common in real files.

```
def normalize_status(status: str) -> str:  
    return status.strip().lower()  
  
for record in reader:  
    status = normalize_status(  
        record["status"]  
    )
```

Raw values

"ACTIVE"
" active "
"Inactive"

Normalized

"active"
"active"
"inactive"

Pipeline habit

Read → convert → normalize → validate → classify → write.

Writing CSV with csv.writer

writerow writes one row; writerows writes many rows.

```
import csv

rows = [
    ["EQ-1001", "ready"],
    ["EQ-1002", "warning"],
]

with open(
    "data/output/results.csv",
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)
    writer.writerow(["record_id", "status"])
    writer.writerows(rows)
```

Output file

```
record_id,status
EQ-1001,ready
EQ-1002,warning
```

Important

Use `newline=""` when writing CSV files. It prevents extra blank lines on some systems.

DictWriter Matches Dictionary Records

When your rows are dictionaries, DictWriter is usually clearer.

```
records = [  
    {"record_id": "EQ-1001", "status":  
    "ready"},  
    {"record_id": "EQ-1002", "status":  
    "warning"},  
]  
  
field_names = [  
    "record_id",  
    "status",  
]
```

```
writer = csv.DictWriter(  
    file,  
    fieldnames=field_names  
)  
  
writer.writeheader()  
writer.writerows(records)
```

Mental model

fieldnames controls the output column order and tells DictWriter which dictionary keys to write.

Validate the CSV Schema First

Before processing rows, confirm the required columns exist.

```
required_fields = {
    "record_id",
    "status",
    "error_count",
}

available_fields = set(reader.fieldnames)

missing_fields = (
    required_fields - available_fields
)

if missing_fields:
    print("Missing columns")
    print(missing_fields)
```

Set difference

$$\begin{array}{l} \text{required_fields} \\ \text{minus} \\ \text{available_fields} \\ = \\ \text{missing_fields} \end{array}$$

Why early?

A missing column is a file problem, not a row problem. Catch it before running the full pipeline.

Mini-Demo: Read, Convert, Classify

A short end-to-end example before exercises.

```
processed_records = []

for record in reader:
    error_count = int(
        record["error_count"]
    )

    status = record["status"].strip().lower()

    if error_count == 0:
        processing_status = "ready"
    elif error_count <= 5:
        processing_status = "warning"
    else:
        processing_status = "rejected"

    processed_records.append({
        "record_id": record["record_id"],
        "status": status,
        "error_count": error_count,
        "processing_status": processing_status,
    })
```

Four operations

- 1 Read row
- 2 Convert number
- 3 Normalize text
- 4 Add classification

Next

The exercises split this demo into smaller steps.

Guided Exercise: 30 Minutes

Students practice reading, transforming, validating, and writing CSV.

1 Read CSV with DictReader

2 Convert numeric fields

3 Normalize status

4 Classify each record

5 Validate required columns

6 Write processed CSV

Timebox: 30 minutes

Exercise Data File

Create data/raw/records.csv with these rows.

```
record_id,status,error_count,completion  
EQ-1001,active,0,100  
EQ-1002,ACTIVE,2,95  
EQ-1003,inactive,0,100  
EQ-1004, active ,7,82
```

Task setup

Keep the source file in data/raw. The output file should go in data/output.

source: records.csv



output:
processed_records.csv

Exercise 1–2: Read and Convert

First prove that rows become dictionaries, then convert numeric text.

```
import csv
from pathlib import Path

input_file = Path("data/raw/records.csv")

with open(input_file, encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for record in reader:
        print(record)
```

Add conversion

```
error_count = int(
    record["error_count"]
)

completion = int(
    record["completion"]
)
```

Timebox: 8 minutes

Exercise 3–4: Normalize and Classify

Use small functions rather than putting every rule in the loop.

```
def normalize_status(status):  
    return status.strip().lower()  
  
def classify_error_count(error_count):  
    if error_count == 0:  
        return "ready"  
  
    if error_count <= 5:  
        return "warning"  
  
    return "rejected"
```

Inside the loop

```
status = normalize_status(  
    record["status"]  
)  
  
processing_status =  
    classify_error_count(  
        error_count
```

Timebox: 7 minutes

Exercise 5: Validate Columns

Detect a bad input file before processing row data.

```
required_fields = {
    "record_id",
    "status",
    "error_count",
    "completion",
}

available_fields = set(reader.fieldnames)
missing_fields = required_fields -
available_fields

if missing_fields:
    print("Missing required columns:")
    print(missing_fields)
```

Test it

Temporarily remove the completion column from a copy of the CSV file. Does your program catch it?

Expected behavior

A schema problem should stop the file pipeline early.

Timebox: 6 minutes

Exercise 6: Write Processed CSV

Produce a new file with an added processing status column.
 data/output/processed_records.csv

```
)

field_names = [
    "record_id",
    "status",
    "error_count",
    "completion",
    "processing_status",
]

with open(
    output_file,
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.DictWriter(
        file,
        fieldnames=field_names
    )
```

Output columns

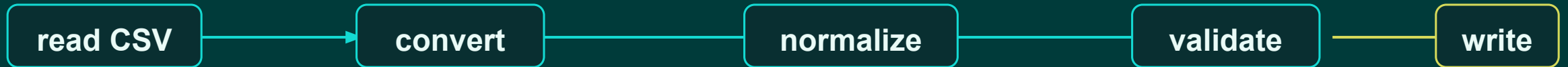
```
record_id
status
error_count
completion
processing_status
```

Timebox: 9 minutes

```
writer.writerow()
writer.writerow(processed_records)
```

Applied Challenge: Equipment CSV Processor

Combine reading, validation, conversion, transformation, and two outputs.



Input

`data/raw/equipment_records.csv`

Outputs

`data/output/clean_equipment_records.csv`
`data/output/rejected_equipment_records.csv`

Challenge Processing Rules

Make the program robust enough for imperfect input.

- 1 Normalize record IDs
- 2 Normalize status
- 3 Convert numeric columns
- 4 Reject invalid numeric values
- 5 Reject missing IDs
- 6 Reject unsupported statuses
- 7 Add processing status
- 8 Write clean and rejected records separately

Challenge principle

Preserve valid source data. Do not overwrite the raw file.

Challenge Scaffold: Split Clean and Rejected

Students fill in normalization, conversion, and validation.

```
clean_records = []
rejected_records = []

for record in reader:
    try:
        processed_record = process_record(record)
    except ValueError as error:
        record["rejection_reason"] = str(error)
        rejected_records.append(record)
        continue

    if processed_record["processing_status"] == "rejected":
        rejected_records.append(processed_record)
    else:
        clean_records.append(processed_record)
```

Goal

Every row ends in exactly one list:

clean_records
or
rejected_records

Instructor prompt

Ask: what should happen when `int("three")` fails?

Block 35 Takeaways

CSV turns rows and columns into Python records.

CSV parser

Use csv module, not split(",")

DictReader

Rows become dictionaries

Strings

Convert numeric columns explicitly

DictWriter

Write dictionary records back out

Schema

Validate required columns early

Pipeline

Read → convert → normalize → validate → write

Next

JSON handles nested structured data and configuration files.